JavaScript

Event Delegation, Debounce & Throttle

Event Bubbling · Capturing · Delegation · Debounce · Throttle · rAF · EventEmitter

PART 1 — DOM Events	Bubbling, Capturing, stopPropagation, preventDefault
PART 2 — Delegation	One listener on parent, e.target, e.currentTarget, closest()
PART 3 — Debounce	Delay until quiet — search inputs, resize, validation
PART 4 — Throttle	Rate-limit — scroll, mousemove, animation, rAF
PART 5 — Advanced	Immediate debounce, trailing throttle, EventEmitter pattern

PART 1 — DOM EVENTS

Event Bubbling & Capturing

Beginner

When an event fires, it travels through the DOM in **3 phases**: Capturing (top → down), Target (the element clicked), and Bubbling (bottom → up). Most listeners use bubbling by default.

```
HTML:


Click Me




PHASE 1 — CAPTURING (top -> down)

window -> document -> grandparent -> parent -> button


PHASE 2 — TARGET

button <-- event fires here


PHASE 3 — BUBBLING (bottom -> up) [default]

button -> parent -> grandparent -> document -> window
```

```
// By default, listeners fire in BUBBLING phase

element.addEventListener('click', handler); // bubbling

element.addEventListener('click', handler, true); // capturing

element.addEventListener('click', handler, {capture:true}); // same
```

1 stopPropagation vs preventDefault vs stopImmediatePropagation

```
// stopPropagation — stops event travelling further up/down

child.addEventListener('click', (e) => {

e.stopPropagation(); // parent never sees this event

console.log('child clicked');

});


// preventDefault — stops browser's DEFAULT action only

// Does NOT stop bubbling!

link.addEventListener('click', (e) => {

e.preventDefault(); // no navigation to href
```

```
});

form.addEventListener('submit', (e) => {

e.preventDefault(); // no page reload

handleFormSubmit();

});


// stopImmediatePropagation — stops bubbling AND other listeners

// on the SAME element

btn.addEventListener('click', (e) => {

e.stopImmediatePropagation(); // nothing else fires on btn

});

btn.addEventListener('click', () => {

console.log('I never fire!'); // same element, blocked

});
```

PART 2 — EVENT DELEGATION

Event Delegation Pattern

Beginner -> Intermediate

> Instead of attaching a listener to every child element, attach ONE listener to the parent and use bubbling to catch all child events — including ones added dynamically.

```
// BAD — listener on every item (expensive, breaks on new items)
document.querySelectorAll('.todo-item').forEach(item => {
item.addEventListener('click', handleClick); // N listeners!
});
// Adding new items dynamically? -> no listener on them!

// GOOD — one listener on parent (delegation)
document.getElementById('todo-list').addEventListener('click', (e) => {
if (e.target.matches('.todo-item')) { // check origin
handleClick(e.target);
}
});
// Works for ALL items — including ones added later!
```

```
HOW IT WORKS

User clicks .todo-item
|
v (event bubbles up)
Event reaches #todo-list
|
v
Our listener fires
|
v
e.target tells us WHICH item was clicked
|
```

```
v

Check if it matches selector -> handle it
```

2 Delegation — Real World: Dynamic List with Multiple Actions

```
const list = document.getElementById('user-list');


list.addEventListener('click', (e) => {


if (e.target.matches('.delete-btn')) { // delete

e.target.closest('.user-item').remove();

return;

}


if (e.target.matches('.edit-btn')) { // edit

const item = e.target.closest('.user-item');

openEditModal(item.dataset.userId);

return;

}


if (e.target.closest('.user-item')) { // row click

selectUser(e.target.closest('.user-item').dataset.userId);

}

});


// New items added later — NO extra listener needed!

function addUser(user) {

list.innerHTML += `

${user.name}

Edit

Delete`;

}
```

3 e.target vs e.currentTarget vs closest()

```
// e.target = element ACTUALLY clicked (event origin)
```

```
// e.currentTarget = element the listener is ATTACHED to


list.addEventListener('click', (e) => {

console.log(e.target); // the or clicked

console.log(e.currentTarget); // always #list

});


// closest() — walks UP the DOM to find nearest matching ancestor

list.addEventListener('click', (e) => {

const item = e.target.closest('.todo-item');

if (!item) return; // clicked outside any .todo-item

console.log('Item ID:', item.dataset.id); // always the right element

});

// Use closest() when nested elements exist inside list items!
```

PART 3 — DEBOUNCE

Debounce — Wait Until They Stop

Intermediate

```
WITHOUT DEBOUNCE (search input):

User types: A -> B -> C -> D -> E

API calls: A B C D E (5 calls — wasteful!)


WITH DEBOUNCE (300ms):

User types: A -> B -> C -> D -> E

Timer resets on each keystroke...

API call: E (1 call after quiet!)
```

```
function debounce(fn, delay) {

let timeoutId;


return function (...args) {

clearTimeout(timeoutId); // cancel previous pending call


timeoutId = setTimeout(() => {

fn.apply(this, args); // schedule new call

}, delay);

};

}


// Usage — search input

const debouncedSearch = debounce((query) => {

fetch(`/api/search?q=${query}`).then(r => r.json()).then(show);

}, 300);


searchInput.addEventListener('input', (e) => {

debouncedSearch(e.target.value); // fires 300ms after user stops

});
```

4 Advanced Debounce — Leading Edge (immediate option)

Sometimes you want to fire IMMEDIATELY on first call (leading edge), then block for the delay period before allowing the next call.

```
function debounce(fn, delay, immediate = false) {

let timeoutId;


return function (...args) {

const callNow = immediate && !timeoutId; // leading edge?


clearTimeout(timeoutId);

timeoutId = setTimeout(() => {

timeoutId = null;

if (!immediate) fn.apply(this, args); // trailing call

}, delay);


if (callNow) fn.apply(this, args); // leading call

};

}


const debouncedImmediate = debounce(handleClick, 500, true);

// 1st call -> fires immediately

// Next calls within 500ms -> ignored

// After 500ms quiet -> 1st call fires immediately again
```

PART 4 — THROTTLE

Throttle — Limit Rate of Execution

Intermediate

```
WITHOUT THROTTLE (scroll event):

Scroll fires: |||||||||||||||||||| (hundreds per second!)


WITH THROTTLE (200ms):

Scroll fires: ||||||||||||||||||

Fn executes: | | | | (once per 200ms)
```

```
function throttle(fn, interval) {

let lastTime = 0;


return function (...args) {

const now = Date.now();


if (now - lastTime >= interval) { // enough time passed?

lastTime = now;

fn.apply(this, args); // execute!

}

// else: too soon — skip this call silently

};

}


// Usage — scroll progress bar

const updateProgress = () => {

const pct = (window.scrollY / document.body.scrollHeight) * 100;

progressBar.style.width = pct + '%';

};


window.addEventListener('scroll', throttle(updateProgress, 100));

// runs at most once per 100ms — no matter scroll speed
```

5 requestAnimationFrame Throttle — Smoothest for Visual Updates

For DOM/visual updates, use rAF instead of time-based throttle. rAF syncs execution with the browser repaint cycle (~60fps), giving the smoothest possible animations with zero jank.

```
function rafThrottle(fn) {
let rafId = null;

return function (...args) {
if (rafId) return; // already scheduled for next frame

rafId = requestAnimationFrame(() => {
fn.apply(this, args); // runs just before next repaint
rafId = null; // ready for next frame
});
};
}

window.addEventListener('scroll', rafThrottle(() => {
updateParallax(); // buttery smooth 60fps
updateStickyHeader();
}));
```

PART 5 — EVENTEMITTER PATTERN

Custom EventEmitter — Pub/Sub

Expert

```
class EventEmitter {

constructor() { this.events = {}; }


on(event, listener) {

if (!this.events[event]) this.events[event] = [];

this.events[event].push(listener);

return this; // chainable

}


off(event, listener) {

if (!this.events[event]) return this;

this.events[event] = this.events[event].filter(l => l !== listener);

return this;

}


emit(event, ...args) {

if (!this.events[event]) return this;

this.events[event].forEach(listener => listener(...args));

return this;

}


once(event, listener) { // auto-remove after first fire

const wrapper = (...args) => {

listener(...args);

this.off(event, wrapper); // remove itself

};

return this.on(event, wrapper);

}

}
```

```
const emitter = new EventEmitter();

emitter.once('connect', () => console.log('Connected!'));

emitter.emit('connect'); // 'Connected!'

emitter.emit('connect'); // nothing — once already fired
```

COMPARISON TABLES

stopPropagation vs preventDefault vs stopImmediatePropagation

Method	Stops Bubbling	Stops Browser Default	Stops Same-Element Listeners
stopPropagation()	Yes	No	No
preventDefault()	No	Yes	No
stopImmediatePropagation()	Yes	No	Yes

Debounce vs Throttle — Quick Decision Guide

	Debounce	Throttle
Fires when	After quiet period ends	At most once per interval
Good for	Search, resize, validation	Scroll, mousemove, animation
Pattern	Resets timer on each event	Ignores until interval passes
Risk	May never fire if non-stop	Always fires at least once
Visual update	Use with care	Prefer rAF for smoother result

INTERVIEW Q & A

Top Interview Questions

Q: What is event delegation and why use it?

A: Attaching one listener to a parent instead of many on each child. Uses bubbling — child clicks bubble up to parent. Benefits: fewer listeners (performance), works for dynamically added elements.

Q: What is the difference between e.target and e.currentTarget?

A: e.target is the element actually clicked (event origin). e.currentTarget is the element the listener is attached to — always the same regardless of which child was clicked.

Q: What is debounce? Give a real use case.

A: Delays execution until after a quiet period. If event fires again before delay ends, timer resets. Use case: search autocomplete — call API only after user stops typing for 300ms.

Q: What is throttle? Give a real use case.

A: Limits how often a function can fire — at most once per interval regardless of event frequency. Use case: scroll handler — update progress bar at most once per 100ms instead of hundreds of times per second.

Q: When would you use requestAnimationFrame instead of throttle?

A: For visual/DOM updates. rAF syncs execution with the browser repaint cycle (~60fps), giving the smoothest possible animations with zero jank. Time-based throttle can fire between repaints causing visual glitches.

Q: What does e.target.closest() do in delegation?

A: Traverses up the DOM from e.target to find the nearest ancestor matching a CSS selector. Essential in delegation when clicking a nested span inside a list item — closest('.item') finds the right parent automatically.

Golden Rules

```
Delegation -> one parent listener catches all child events via bubbling.

e.target -> what was clicked. e.currentTarget -> where listener lives.

Debounce -> wait until QUIET. Best for: search, resize, validation.

Throttle -> fire ONCE per interval. Best for: scroll, drag, animation.

rAF -> throttle for visuals. Syncs to 60fps browser paint cycle.
```